THE ENVELOPE

Why the ATLAS envelope is the right spine for CryptAlyst

architecture note · no code · decision document

Source	What was read
ATLAS envelope design	ENVELOPE-OVERVIEW, ENVELOPE-DESIGN, THE-ENVELOPE-BOOK, RFC-0001
ATLAS implementation	internal/datax/envelope.go — Envelope[T], Meta, Transformer, Pipe, Decode
ATLAS bus	ATLAS_BUS.md — topics, JSONL journal, replay, slow-consumer drop
ATLAS storage	storage-agnostic carrier: consumers never see a store (journal entry 2026-08-23)
CryptAlyst	the running platform: Go feed, RabbitMQ bus, 30 Python model books

This document argues one claim: the envelope is not a serialisation format, it is the mechanism that makes market data first-class, indicators auditable, and backtests identical to live. CryptAlyst currently has none of those three properties, and every one of them is a consequence of the same missing abstraction.

1 What CryptAlyst is, and what is wrong with it

CryptAlyst is a paper-trading platform: a Go market-data service streaming 161 websocket channels, a RabbitMQ bus carrying nineteen topics, thirty Python model books, and a dashboard. It works. It is also, architecturally, a set of services that happen to share a broker.

Six defects matter, and they share a root cause.

Defect	Symptom today	Root cause
No priority class	depth bursts compete with prices for the same connection and pool	one topic space, no plane separation
Indicators are not first-class	models publish decisions and fills, never the thing they computed	no advisory contract
No provenance	an engine cannot say which inputs produced a decision	messages carry no lineage
No augmenting	each consumer recomputes what another already computed	messages are immutable end-states
No replay	backtests read REST candles and differ from live by construction	nothing is recorded as it was consumed
No freshness contract	a forty-minute-old forecast looks identical to a four-second-old one	no generation or computed_at in the payload

All six are the same absence: there is no shared object that carries data, says how fresh it is, says what it was made from, and can be extended as it travels. ATLAS built that object and called it the envelope.

2 What the envelope is

The envelope is a fixed shape that every request and every answer travels in. It carries three things and nothing else: the data, metadata about that data, and references to data it does not carry. Because the shape never changes, new requirements do not reopen old code — the behaviour lives around the envelope, not inside it.

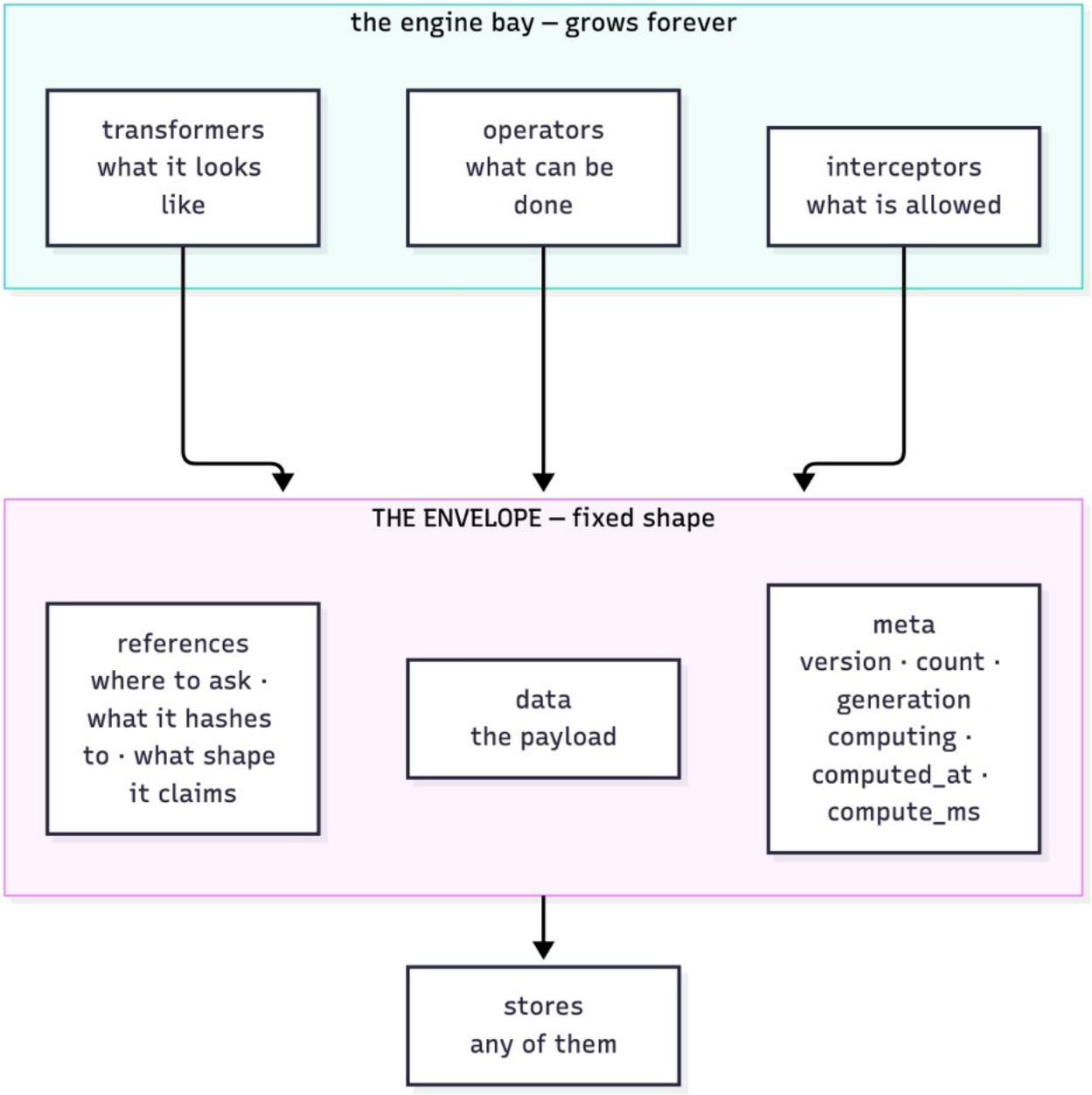


Figure 1 — Fixed shape, growing engine bay. The envelope is the plug; everything else is the appliance.

2.1 The parts

Part	Role	In CryptAlyst terms
data	the payload	candles, ticks, flow, forecast, decision
meta	provenance and freshness: version, count, generation, computing, computed_at, compute_ms	was this computed from the tick I think it was?
references	where to ask, what it hashes to, what shape it claims	the tick batch behind a flow reading
operator	advertises a capability, produces the result	a calculator or a model

interceptor	runs before work, can refuse with the rule named	the risk governor
transformer	changes the shape of an answer, never its truth	dashboard projection, redaction

2.2 The twelve principles, and the one that matters most here

ATLAS states twelve principles. For a trading platform the decisive ones are:

- P2 — the shape never changes: a new model is an appliance, not a protocol change.
- P4 — one identity: a correlation id follows a decision from tick to fill.
- P5 — refer, do not copy: an envelope points at the tick batch instead of embedding it.
- P7 — evidence is an output: the audit record is produced by the work, not typed afterwards.
- P11 — honest metadata: the use-by date is on the label, so an engine can refuse stale input.
- P12 — incremental forever: fixed bricks, infinite builds.

2.3 The lifecycle: append, monitor, satisfy

The envelope is not a packet that is sent and dies. It travels, being monitored, until the minimum requirements are satisfied. Work that can help appends its step and a correlation id; the envelope accumulates provenance as it goes.

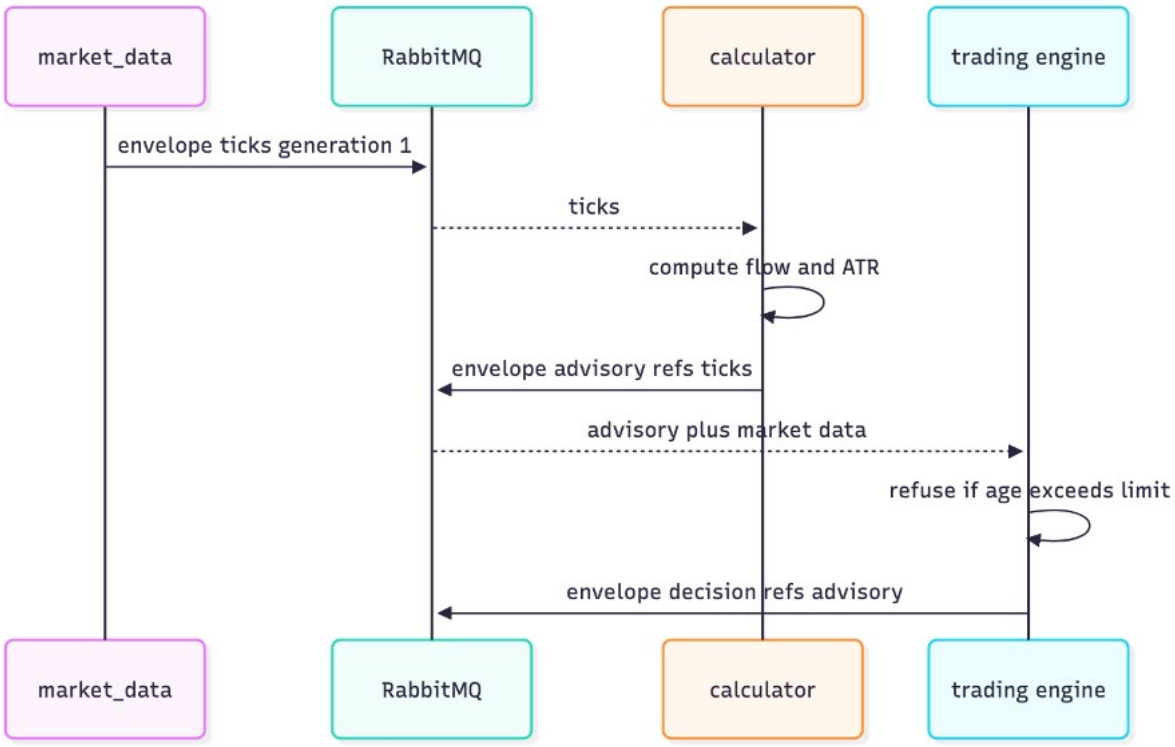


Figure 2 — One decision, three envelopes. Each stage appends provenance; none changes the shape.

This is precisely the shape of a trading decision: market data arrives, calculators augment it with indicators, the orchestrator decides what runs, engines act, risk refuses or permits, and the whole chain can be reconstructed afterwards from the identities the envelope collected.

3 The three layers this adds to CryptAlyst

3.1 The envelope — the carrier

Every message on the bus becomes an envelope. Market data publishes envelopes whose meta carries the generation of the book that produced them. Calculators publish envelopes whose meta carries references to the envelopes they consumed. Engines publish decisions whose meta references the advisories used. Nothing about the shape changes as the platform grows; models add behaviour, not contract.

3.2 The orchestrator — what runs, where, and when

ATLAS uses an orchestrator as the component that reads envelope metadata and decides what work happens next. In CryptAlyst this replaces a static supervisor that restarts processes on a timer.

Orchestrator responsibility	Today	With the envelope
What to compute	every model runs on its own clock	a calculator declares a capability; the orchestrator asks for it when an envelope needs it
When to recompute	fixed loop intervals	meta.generation and meta.computed_at say whether the answer is current
What to restart	process liveness only	a consumer whose envelope lag exceeds its contract is rescheduled
What to persist	each book writes its own JSON	the orchestrator routes envelopes to the recorder; storage is not the model's business
What is allowed	risk checks inline in each model	an interceptor refuses before the work starts, with the rule named

The critical property: the orchestrator never knows what a model computes. It knows capabilities, deadlines and metadata. That is what makes adding the thirty-first model a registration rather than a refactor.

3.3 The agnostic persistence layer — write once, read anywhere

ATLAS's storage-agnostic carrier is explicit: handlers and consumers see envelopes, never a store. The same envelope can be persisted to memory for the hot path, to a columnar store for research, and to an object store for cold retention, without a consumer knowing which.

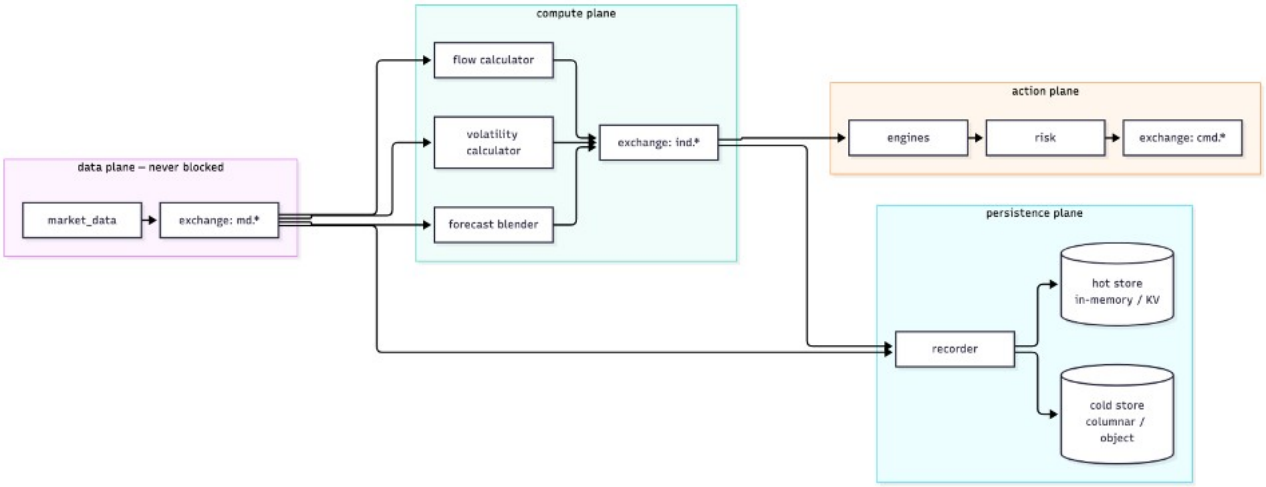


Figure 3 — Four planes. Market data has its own; persistence is a consumer, never inline.

For CryptAlyst this is the difference between the current state and research capability. Today the record of a decision is a JSON line a model wrote about itself. With an envelope, the record is the envelope chain: what

arrived, what was computed from it, what was decided, what was refused and why. That is the dataset a backtest needs in order to be a backtest rather than a hypothesis.

4 What the envelope fixes, one defect at a time

4.1 Market data priority becomes a property of the object

Today priority is a hope. Depth and prices are separate topics but share a broker, a connection pool and a consumer pool, so a burst of depth work can delay a price delivery.

With envelopes, priority is carried in meta and enforced by topology: the market-data plane has its own exchange, its own queue family and its own consumer budget. An envelope that says plane=market is delivered by a pool that nothing in the compute plane can starve. The recorder is a consumer, not a hook — the feed never waits for a disk write, because the write is somebody else's envelope.

4.2 Indicators become a contract

The calculator plane publishes advisory indicators, not outcomes. An advisory envelope carries the value, the horizon it applies to, the confidence, the expiry, and references to the inputs it was computed from. An engine that acts on it can say, precisely, what it acted on.

```
envelope(advisory)
  data:  { symbol: BTCUSDT, kind: flow, value: +0.42, horizon_s: 60 }
  meta:  { version: 1, generation: 8834, computed_at: 09:41:02.114, compute_ms: 3 }
  refs:  [ envelope(ticks)@8834, envelope(candles_1m)@8834 ]
```

This single change makes three current problems disappear: duplicated computation (the second consumer reads instead of recomputes), unauditable decisions (the refs are the audit), and the stale-input class of bug (the computed_at is on the label).

4.3 Backtesting becomes replay

ATLAS records every event to an append-only journal and replays it on startup, so a subscriber can resume from an offset. CryptAlyst records almost nothing and replays nothing, which is why its backtest harness reads REST candles and produces numbers that do not correspond to live trading.

With the envelope and an agnostic recorder, the platform stores the exact envelopes it consumed. A backtest then feeds those envelopes back through the same calculators at simulated speed. Live and backtest run the same code against the same inputs — the only difference is the clock. That is the difference between measuring a strategy and inventing one.

4.4 Freshness becomes enforceable instead of aspirational

CryptAlyst already has staleness gates: a forecast older than 1,200 seconds is refused, a signal older than 90 seconds is refused. They work because the engine guesses age from a wall clock. With the envelope the guess disappears: meta.computed_at and meta.generation are the truth, and a refusal can quote them.

```
refused: advisory generation 8834 older than the tick generation 8841
refused: computed_at 09:12:44, age 1,706s, limit 1,200s
```

5 Why this is a good fit — and why now

5.1 The fit, stated plainly

CryptAlyst requirement	Envelope mechanism	Consequence
Market data always first	plane in meta + separate topology	priority is enforced, not hoped for
Many consumers, one computation	advisories as envelopes with refs	compute once, read many
Auditability of a decision	one identity, refs, append as it travels	every decision reconstructable
Persistence for research	storage-agnostic carrier	hot and cold stores from the same object
Backtest equals live	journal + replay of envelopes	same code, same inputs, simulated clock
Refusal with a reason	interceptor before the work	rules named, decisions recorded
Adding a model without a refactor	operators advertise capabilities	registration, not integration

5.2 Why the code already looks like this, and why that matters

The ATLAS implementation is small and severe: an Envelope[T] with Data and Meta, a Transformer that changes an envelope in flight without mutating shared references, a Pipe that composes transformers, and a Decode that refuses a top-level member the contract does not define while ignoring unknown meta keys. The version policy accepts the current revision and the one before it, and refuses anything newer rather than misreading it.

That strictness is the reason it will survive contact with a trading platform. The payload is validated; the metadata is extensible. New provenance can be added without breaking readers, and a malformed payload cannot silently become a trading decision.

5.3 What it costs, stated honestly

- Discipline: every publisher must produce an envelope, including the Go services that currently publish bare JSON.
- Storage: recording every envelope is more data than recording fills. It needs retention policy from day one.
- Latency: an envelope is larger than a bare payload. At 4 snapshots per second this is irrelevant; at per-tick rates it is not, which is why market data uses batched envelopes.
- Versioning: schemas must be versioned before there are two producers, not after.
- Migration: the existing thirty books publish model.decision and model.fill in a shape that must be wrapped, not rewritten.

6 The two services that make it real

The envelope is the carrier. Two services make it useful at platform scale: a correlation service that turns a trace into a queryable record, and a data lake that stores every envelope once and serves it from whichever tier is cheapest. CryptAlyst has neither, which is why its decisions cannot be explained and its history cannot be replayed.

6.1 The correlation service — identity, mapping, gated resolution

ATLAS states its whole design in one line, and it is the line that decides whether the service can be down without stopping the platform:

```
minting is local      a cid is created at the boundary that created the work
mapping is central    someone must own who is doing this for whom
resolution is gated   reading the map is privileged, and every read is recorded
```

That asymmetry is deliberate. Work still starts, promises are still made, fulfilments still append when the service is unavailable — only introduction and resolution wait. It is valuable without being a single point of failure for execution, which is the property a trading platform needs from anything on the critical path.

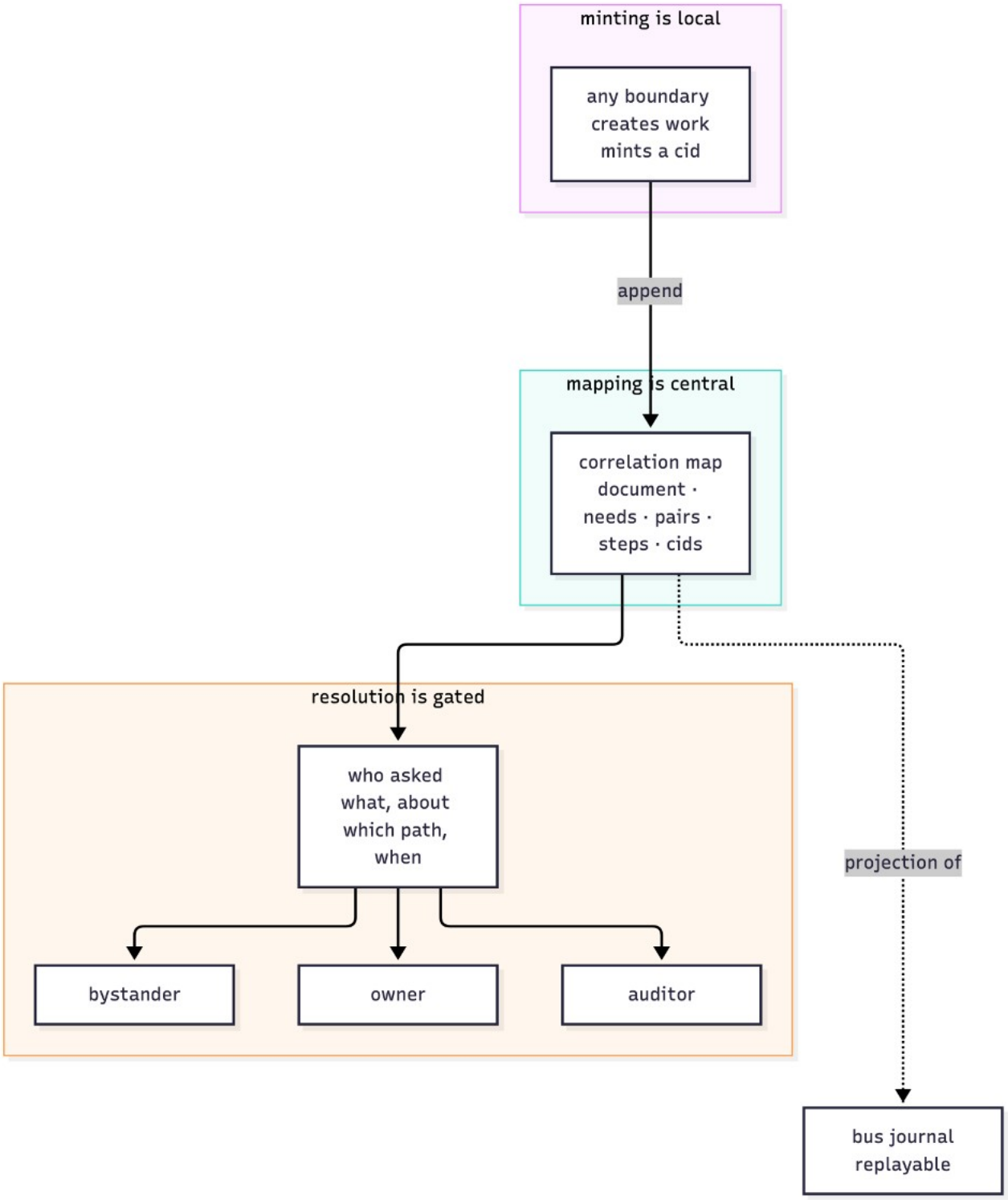


Figure 14 — Mint anywhere, map centrally, resolve under authority. The map is a projection of the journal and can be rebuilt by replay.

The map itself is a projection of durable records — the bus journal, the ledger, the capability registry — never the source of truth. That is what allows a restart to be a non-event and more than one instance to exist. For CryptAlyst the projection is the same journal it already writes: every envelope appends a step, and the map is the view that answers questions about those steps.

Service job	ATLAS mechanism	CryptAlyst equivalent
Introduce	bring a need and an able actor into contact	orchestrator pairs a calculator capability with an envelope that needs it
Pair	record which fulfilment satisfies which	which advisory satisfied which engine's

	obligation	request
Resolve	answer what happened for a cid, subject to authority	why did we buy BTCUSDT at 09:41

Two rules from the design are worth importing verbatim, because they are what keep a trace trustworthy:

- Every output of the service is either a step on an existing envelope or a new envelope. There is no private API that bypasses the contract.
- Resolution is recorded on the document being read, so reading the trace becomes part of the trace.

The second rule answers a question a trading desk will ask on day one: who looked at this decision, and when? A resolver without that property is an audit hole dressed as an audit tool.

6.2 The agnostic data lake — one API, many tiers, no consumer knows which

ATLAS's lake is the other half of why the envelope works at scale. The problem it solved is exactly the problem CryptAlyst has: a single JSON map that grew to two gigabytes, where every browse was a full scan and every aggregate was recomputed at read time. The fix was not a faster query — it was a tiered, sharded, pluggable store behind one envelope API.

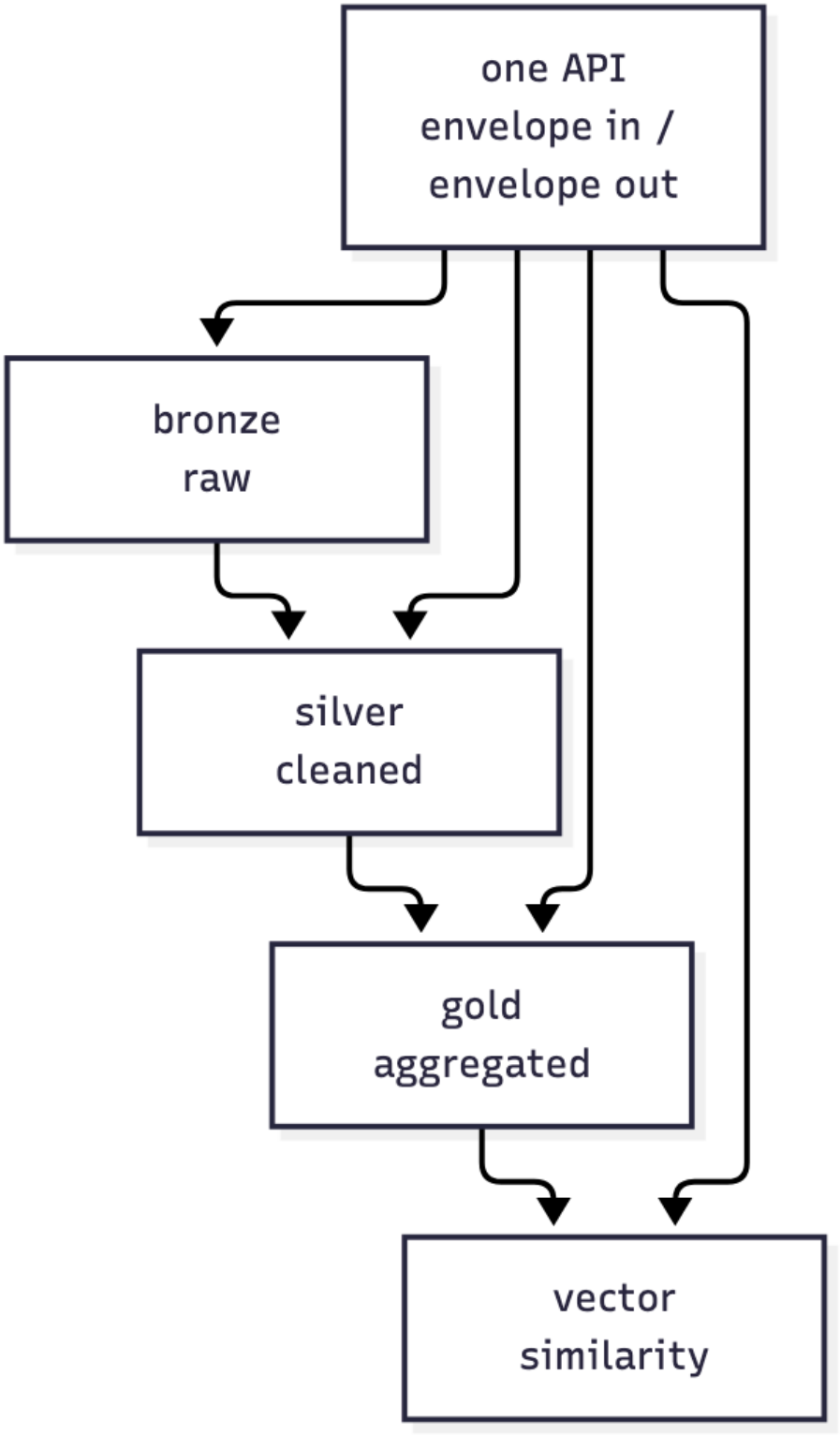


Figure 15 — One API, four tiers, each backend replaceable. The consumer asks and receives; it never learns which tier answered.

Lake property	What it means	Why CryptAlyst needs it
Tiered	bronze raw, silver cleaned, gold aggregated, vector for similarity	raw ticks for replay, bars for research, advisories for the dashboard
Sharded	one shard per root or per symbol class, opened lazily	tick history per symbol without one giant file
Pluggable	every tier behind one Backend interface	memory today, columnar tomorrow,

		without touching consumers
Write-behind coalescing	last-write-wins for dirty records	book equity updates at high frequency without a write per tick
Scatter-gather	worker-capped fan-out across shards	a cross-section rank over 40 symbols without blocking the feed
One API	envelope in, envelope out	the same object the bus carries is the object the lake stores

The important consequence for a trading platform is not speed, it is equivalence. If the lake stores the envelopes the bus carried, then a backtest reads the same objects the live system consumed. Nothing has to be re-derived, and no research result can be an artefact of a different data path.

6.3 What the two services together give CryptAlyst

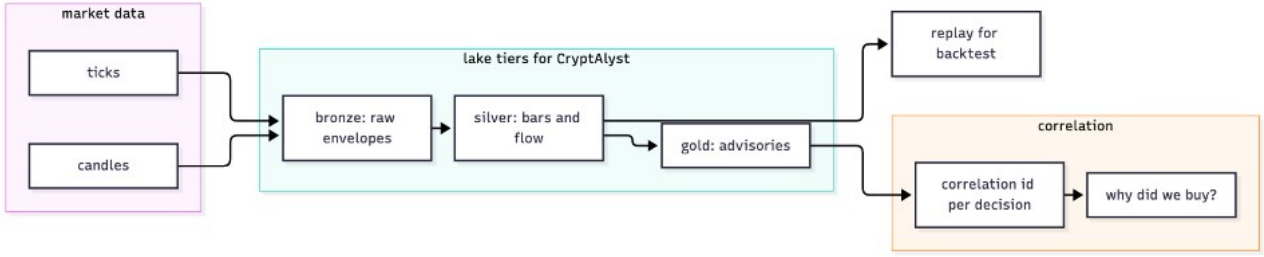


Figure 16 — Ticks into bronze, bars and flow into silver, advisories into gold, a correlation id per decision, replay from the lake.

With both in place the platform can answer three questions it cannot answer today: what happened, why it happened, and what would have happened. The first two come from the correlation service; the third from replaying the lake through the same calculators. Today the first is a log file, the second is an opinion, and the third is a different program.

The ordering matters. The lake without the correlation service gives searchable history with no lineage. The correlation service without the lake gives lineage that cannot be replayed. Together they are the difference between a trading platform and a research programme that trades.

7 Ideas, and the use cases they unlock

Each idea below is one property of the envelope applied to a concrete problem CryptAlyst has today. The diagrams are deliberately small: one idea, one consequence, one thing that stops being a special case.

7.1 Idea: one shape, two planes

Market data and everything derived from it are different kinds of traffic with different failure modes. The envelope lets the platform say so in metadata, and the topology enforces it.

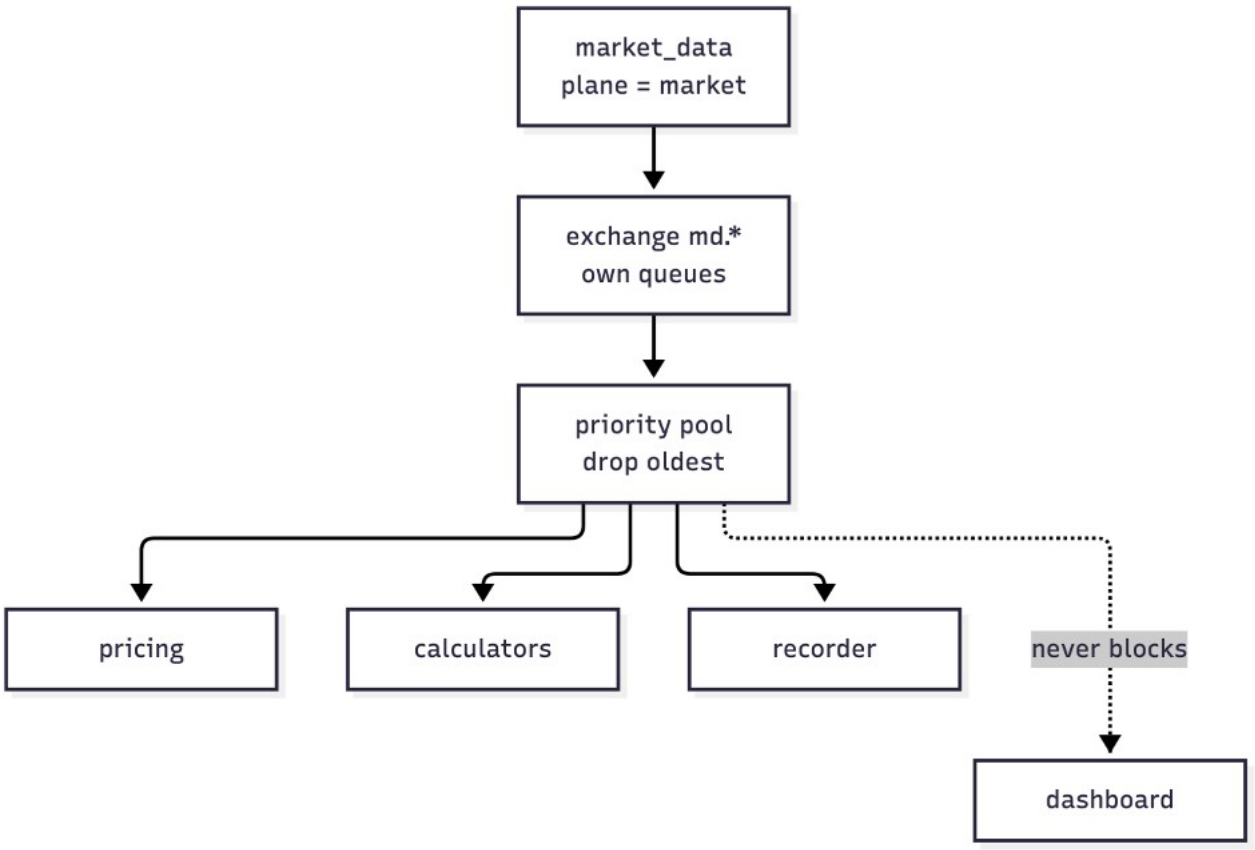


Figure 4 — The market-data plane has its own exchange, queue family and consumer budget. The dashboard is a hint, never a blocker.

7.2 Use case: compute once, read many

Today three models recompute the same flow reading. With advisories, the calculation happens once and the result travels with its inputs attached.

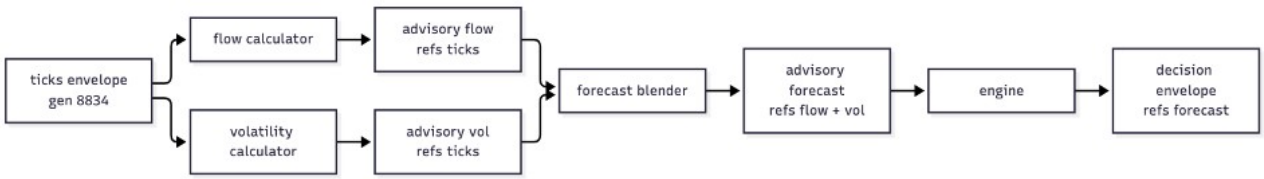


Figure 5 — Two calculators, one blender, one engine. The forecast references flow and volatility rather than re-deriving them.

7.3 Idea: the twelve principles, mapped to trading

ATLAS states twelve principles. Four of them do almost all the work in a trading system: one shape (a new model is an appliance), one identity (a decision can be traced), refer-don't-copy (the tick batch stays in one place), and honest metadata (freshness is on the label).

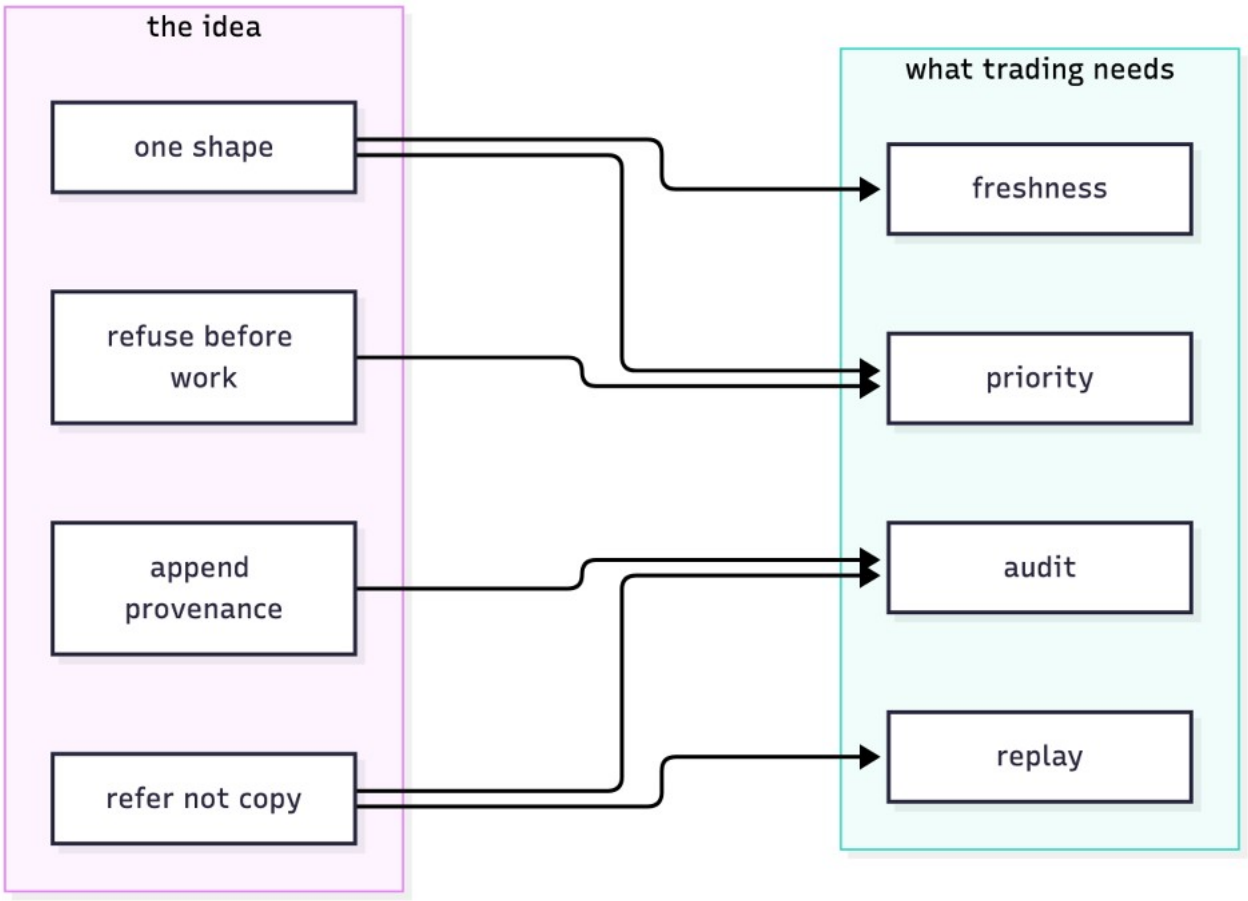


Figure 6 — The ideas on the left, the properties a trading platform actually needs on the right.

7.4 Use case: why did we buy?

This is the question a reviewer asks after a bad day, and today CryptAlyst cannot answer it. With references, the decision envelope points at the advisories, which point at the ticks. One query, one chain.

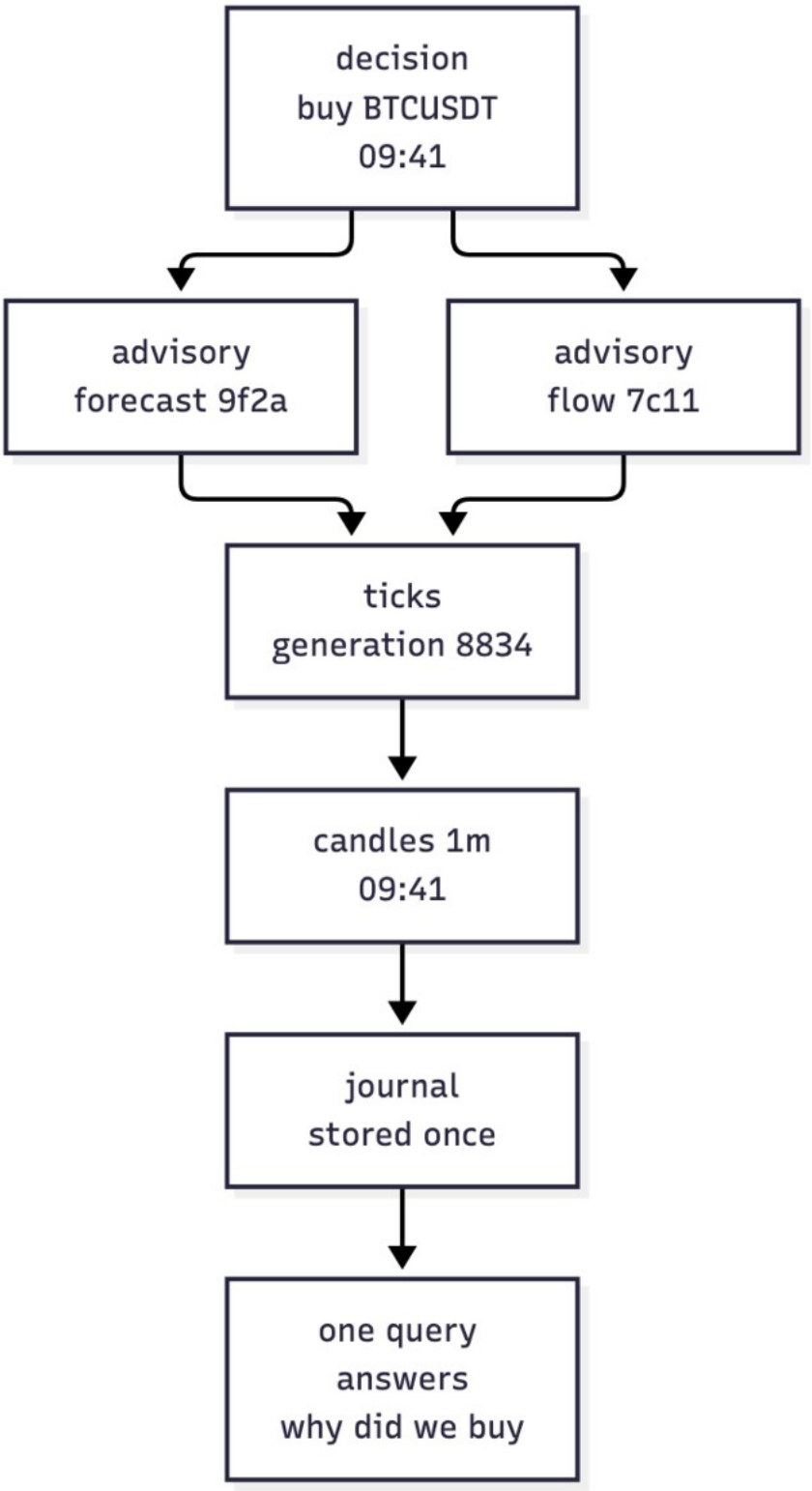


Figure 7 — A decision resolved to the candle that produced it. Every link is a stored envelope, not a log line.

7.5 Use case: the same code in live and in backtest

The weakest part of the current platform is that the backtest and the live system share intent but not code paths. Record the envelopes, replay them, and the difference between live and simulated disappears except for the clock.

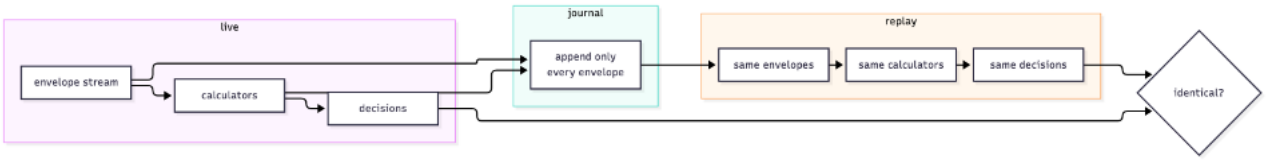


Figure 8 — Journal on the left, replay on the right, identical envelopes in between. The comparison is the test.

7.6 Idea: the orchestrator dispatches capabilities, not processes

A supervisor restarts processes. An orchestrator asks for a capability and gets an answer with metadata attached. That is the difference between knowing a service is alive and knowing its answer is current.

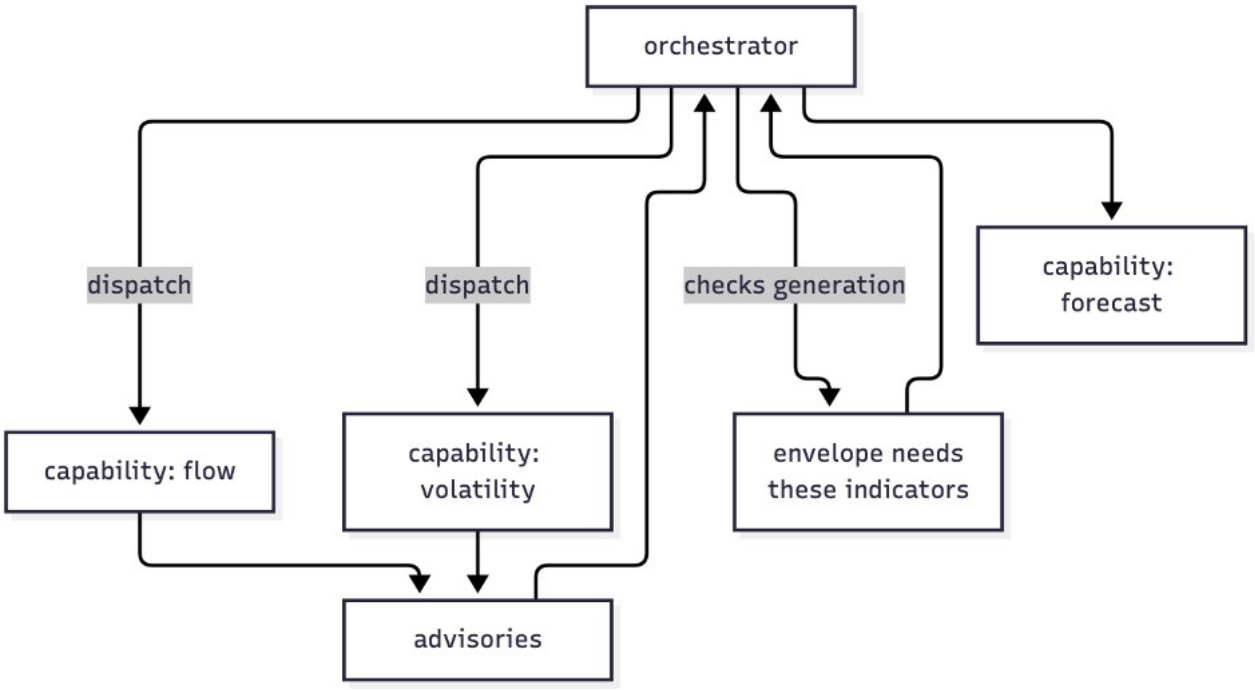


Figure 9 — The orchestrator never knows what a calculator computes; it knows what it can do and whether its answer is fresh.

7.7 Use case: one envelope, three stores

Research wants columnar history, the hot path wants memory, compliance wants an archive. A storage adapter means the platform writes one object and reads it from wherever it lives.

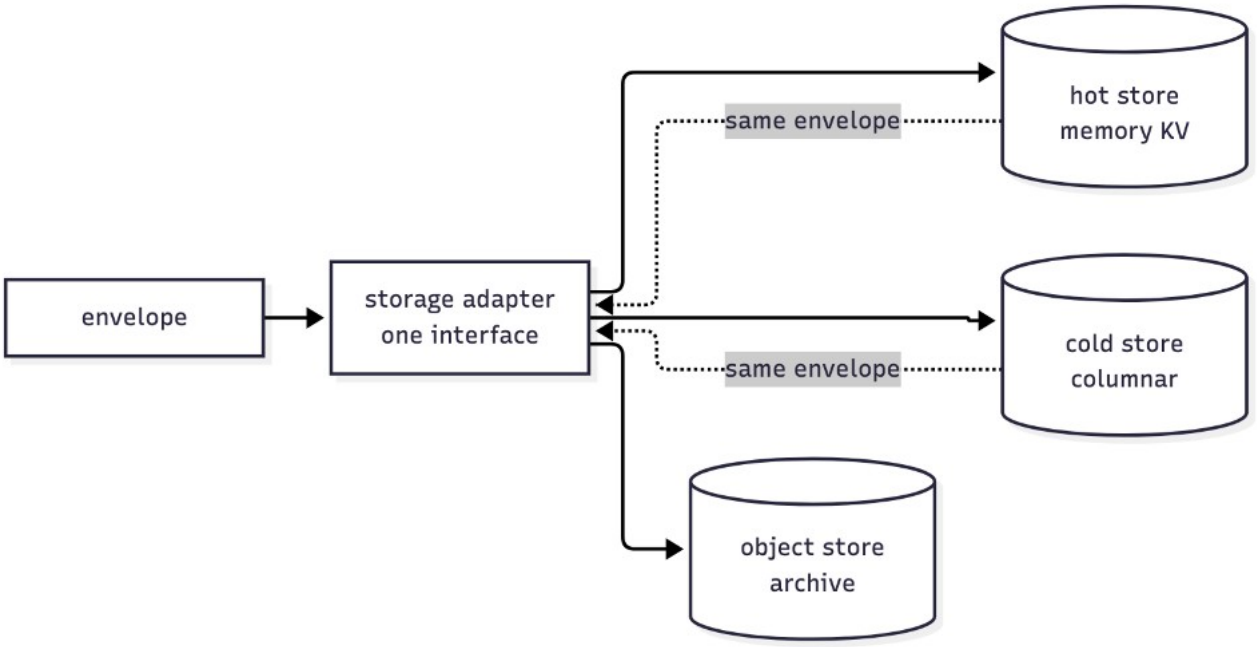


Figure 10 — Same envelope, three adapters. No consumer knows which store answered.

7.8 Use case: refusing stale input with evidence

CryptAlyst already refuses stale forecasts, but it refuses on a wall-clock guess. With meta the refusal can quote the envelope: the generation it came from, when it was computed, and the limit it breached.

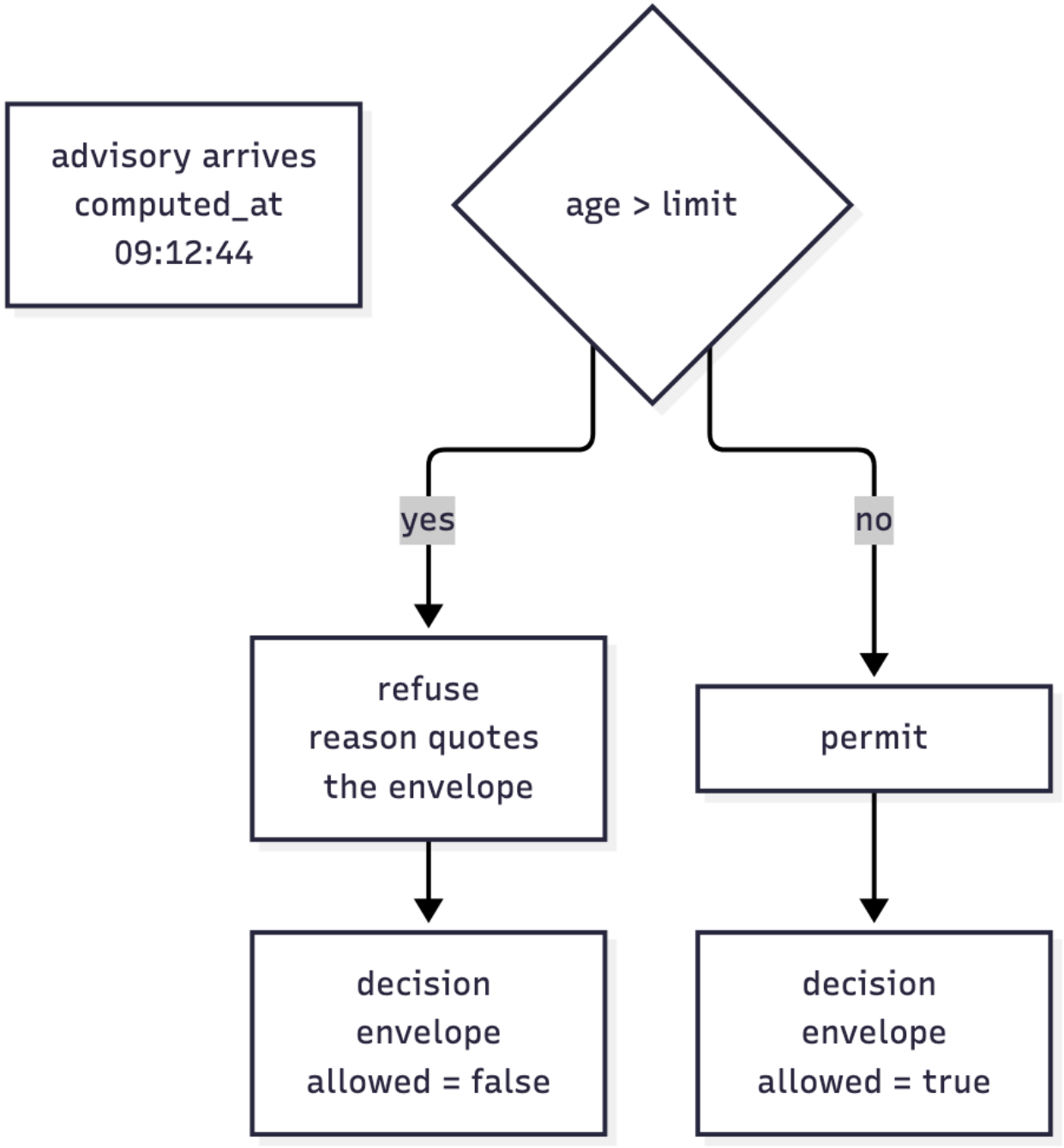


Figure 11 — Refusal is a decision envelope too, and it carries the evidence that caused it.

7.9 Idea: thirty books without a dependency tangle

Thirty models sharing four inputs is a graph, and today that graph exists only in the code that fetches. With envelopes it is explicit in the subscriptions, and therefore inspectable.

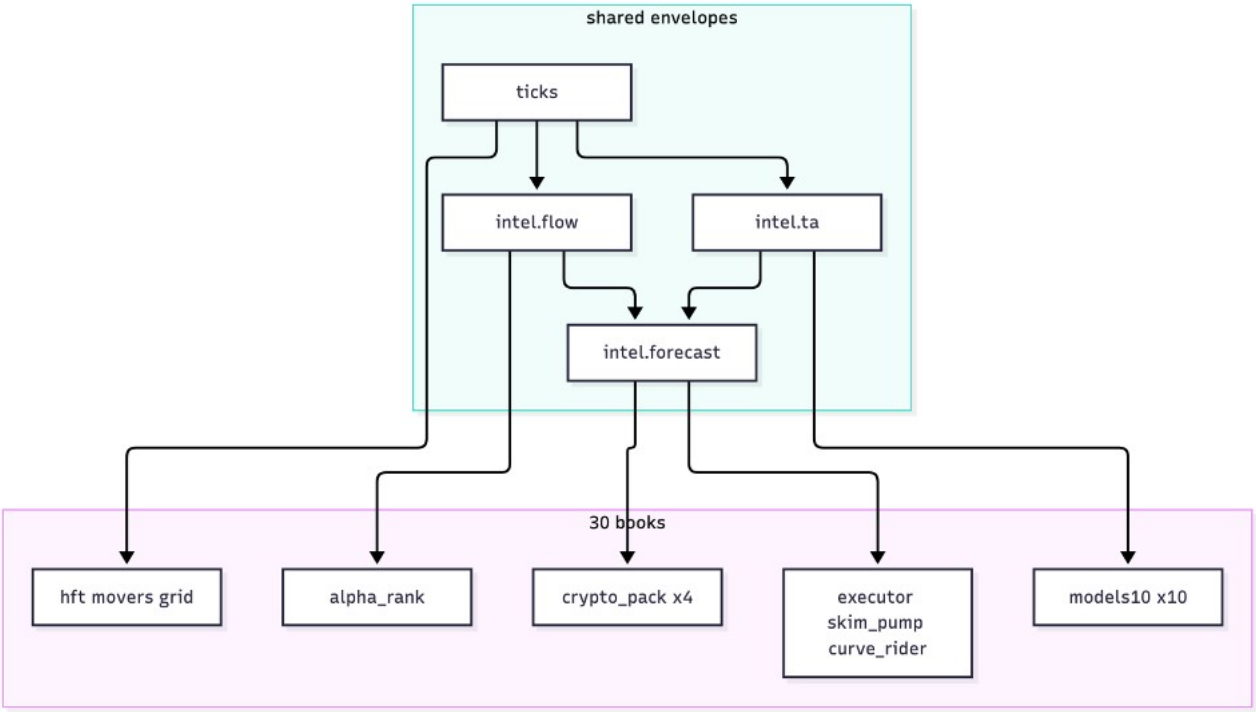


Figure 12 — Four shared envelopes feeding thirty books. Adding a book adds an edge, never a code path.

7.10 Use case: a new model is a registration

The test of the architecture is what happens when the thirty-first model arrives. If the answer involves editing a service, the abstraction failed.

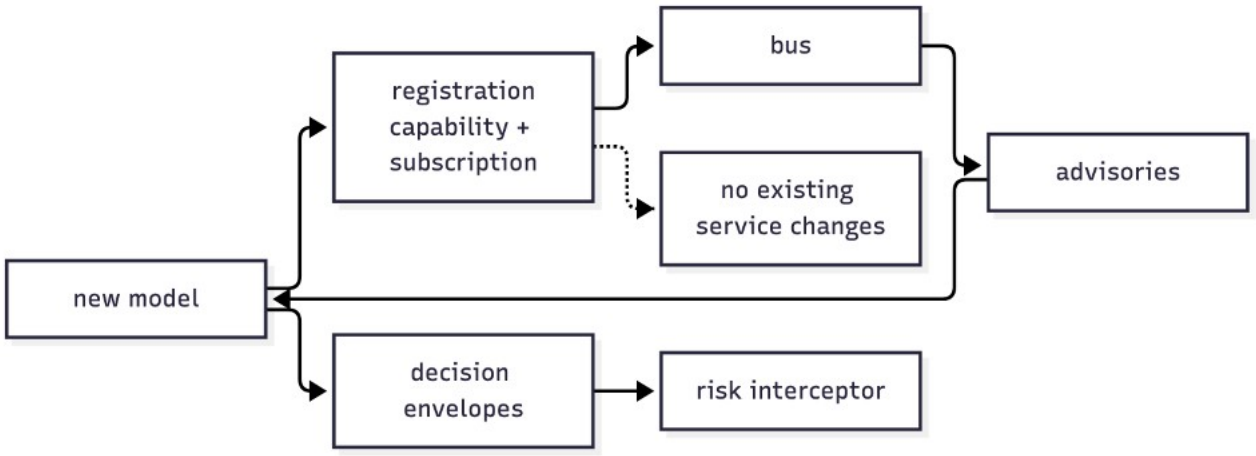


Figure 13 — Registration, subscription, decisions. Nothing existing changes.

8 How to adopt it without a rewrite

The order below follows the migration already agreed for the Go services, and each phase leaves the platform working.

Phase	Change	Proof it worked
1	Define the envelope in Go; wrap the four Go services (market data, consumer, health, alerts)	every topic carries data + meta + refs; version policy enforced
2	Recorder service: one consumer, writes envelopes to a hot store and a cold journal	replay a five-minute window and reproduce the same prices
3	Calculator tier: flow, volatility, regime publish advisory envelopes	advisories carry refs to the ticks they used
4	Orchestrator: capabilities, deadlines, generation checks	a stale advisory is not recomputed but replaced
5	Engines: consume advisories, refuse on meta freshness, emit decision envelopes	every decision names the advisories behind it
6	Backtest: replay recorded envelopes through the same calculators	live and backtest agree on the same window

8.1 The one invariant to protect

The envelope never changes shape. Everything that varies — venues, indicators, models, stores — lives in operators, interceptors and transformers around it. The moment the envelope grows a trading-specific field, the design has failed and the platform is back to a bespoke message per consumer.

That is the whole argument. CryptAlyst does not need another bus or another model. It needs one object that carries data, says how fresh it is, says what it was made from, and can be extended as it travels. ATLAS already built it, hardened it, and proved it in a domain with stricter referees than ours.